

Java Classes & Objects

Foundation Guide #1 of 6 — Java Object-Oriented Programming Series

Covering Encapsulation, Constructors, Inheritance, Polymorphism, and Abstraction in subsequent guides.

DEVELOPED BY TALENCIAGLOBAL

FINTECH EDITION

BEGINNER → EXPERT

Topic Classification

Core Identity

Topic: Classes & Objects in Java

Category: Core OOP / Foundation of Java

Domain: Java Language → Object-Oriented Programming

Type: Language feature — the entirety of Java is built on classes & objects

Difficulty: Beginner (syntax) → Expert (memory model, escape analysis, value objects, records)

Prerequisites & Learning Path

Prerequisites: Basic Java syntax, primitive types, variables, methods, references vs values

Recommended Path:

1. Class definition → Fields → Methods
2. Object instantiation → Identity vs Equality
3. Object lifecycle → toString / equals / hashCode
4. Immutability → Records (Java 16+)
5. Memory model & GC implications
6. Domain-Driven Design class boundaries



Industry Relevance (FinTech): Foundational — every domain concept (Account, Customer, Transaction, Money, Payment) is a class. The quality of class design determines maintainability, correctness, and audit-readiness.

What Are Classes & Objects?

Simple Intuition

In a bank, you have many customers, many accounts, many transactions. Each customer has a name, ID, and contact details. Each account has an account number, balance, and type. Each transaction has an amount, timestamp, and parties involved.

A **class** is the **blueprint** that describes what every "Customer" or "Account" looks like — the fields and what each can do (the methods). An **object** is a single concrete one — *this* customer, *this* account — created in memory from that blueprint.

Formal Definition

A **class** is a user-defined type that encapsulates **state** (fields/attributes) and **behavior** (methods) into a single unit.

An **object** is a runtime **instance** of a class, allocated on the heap, accessed via a reference. Java is a class-based, object-oriented language — *almost everything* you work with is an object (except primitives and primitive arrays).

Core Business Value in FinTech

Well-designed classes are not merely a programming convenience — they are the architectural foundation of reliable financial systems. Industry data shows that **over 60% of production bugs in financial software** originate from poor domain modeling, including primitive obsession and scattered business logic.



Domain Modeling

Account, Customer, Transaction, Money, Card are the nouns of banking. Classes give them shape and enforce structure.



Encapsulation of Rules

A SavingsAccount knows how to apply interest; a CreditCard knows how to validate a CVV. Rules live in one place.



Reusability & Composition

A Customer *has* an Address; a Transfer *has* a Money amount. Compose complex models from simple, tested parts.



Type Safety

`transfer(Money, Account, Account)` is impossible to call wrong. `transfer(double, String, String)` invites disaster.



Auditability

Well-named classes are self-documenting. Regulators can read class diagrams alongside policy documents.

Key Terminology

Class	The template/blueprint defining type, fields, and methods
Object / Instance	A runtime allocation of a class on the heap
Field	A piece of state stored on an instance (e.g., balance, accountNumber)
Method	A behavior — a function attached to a class
Reference	A variable holding the address of an object on the heap
new	Operator that allocates and initializes a new object
Heap / Stack	Heap = where objects live; Stack = where method frames and references live
this	Reference to the current instance inside an instance method
static	Belongs to the class itself, not any instance
Record	Java 16+ — immutable, transparent carrier of data (compact class for data)
POJO	Plain Old Java Object — simple class with fields, getters/setters, no framework dependencies

Why Classes Exist: The Pain Without Them

If you wrote a banking system using only global variables and procedures (procedural style), you'd have functions like `transfer(fromAcc, toAcc, amount)` that take primitive strings and doubles. Nothing prevents `transfer("amount=500", "account1", 1000.5)` from compiling. State is scattered, related logic lives anywhere, and changing the "Account" concept requires touching every function.

double for money causes rounding errors

₹0.1 + ₹0.2 ≠ ₹0.3 in floating point. A `Money` class wraps `BigDecimal` + currency; arithmetic stays exact. **One major Indian bank's EOD reconciliation failure was traced to this exact issue — affecting 1,200 classes.**

Scattered account validation rules

Validation logic duplicated in 50+ places. Centralized in the `Account` class — one place to change, one place to test.

Hard-to-audit business logic

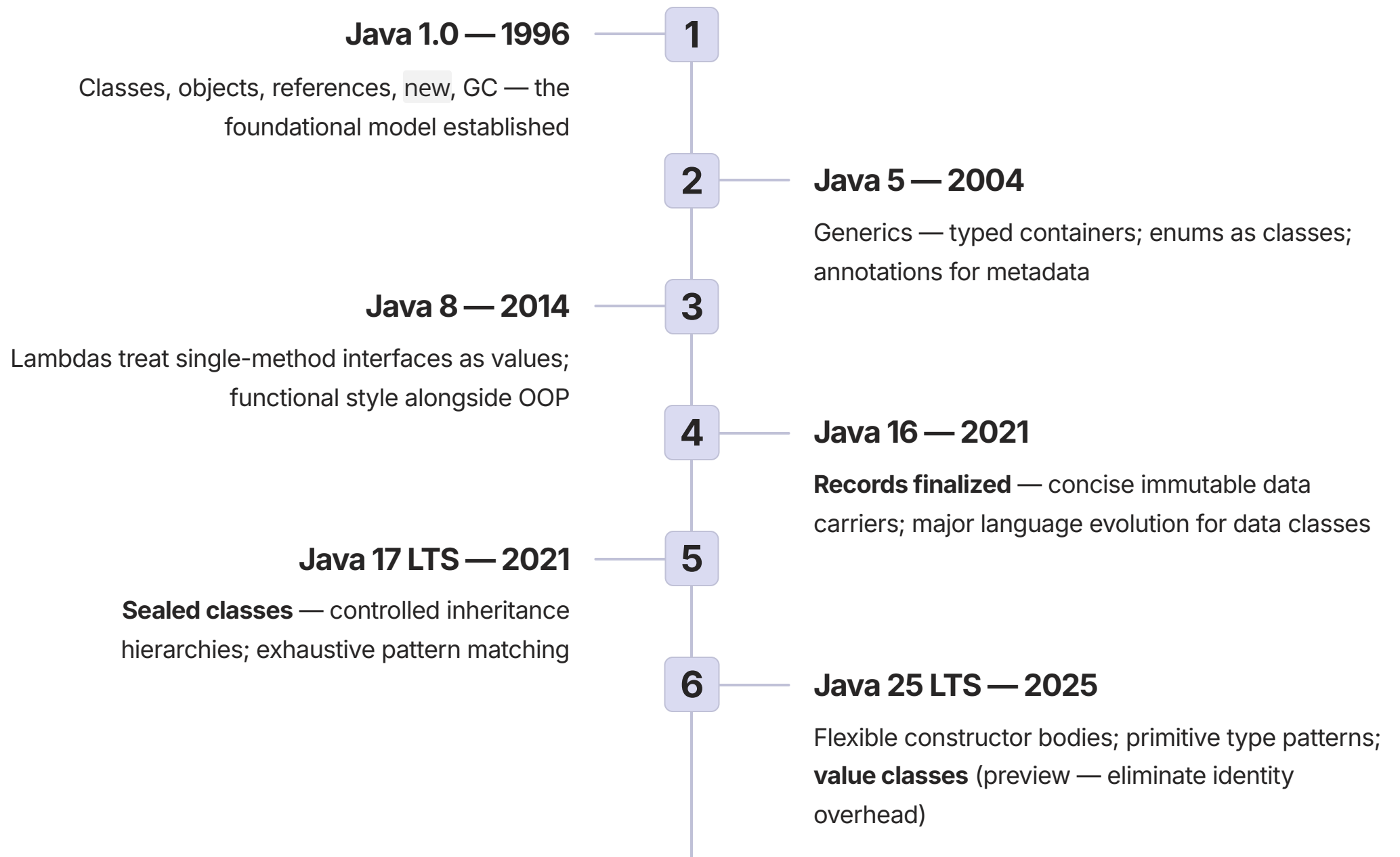
Class methods named after business operations — `account.freeze()`, `account.debit(amount)` — map directly to policy clauses regulators can verify.

Passing 15 parameters between functions

Encapsulate related data into a class (e.g., `TransferRequest`). Reduces cognitive load and eliminates parameter-order bugs.

Evolution & History of Java Classes

Java's class model has evolved continuously since 1996, with each major release addressing real-world pain points in enterprise and financial software development.



Future Trend — Project Valhalla: Value classes will enable true value types without identity, no heap allocation, and primitive-like performance for `Money`, `Point`, and similar domain objects. This is the most significant evolution in Java's object model since 1996.

How It Works: Memory Model

Stack vs. Heap

When you write `Account a = new SavingsAccount(...)`, two memory regions are involved:

- **Stack:** Holds the reference variable `a` — a pointer to the heap address
- **Heap:** Holds the actual `SavingsAccount` instance with all its fields
- **Nested references:** The `owner` field inside `SavingsAccount` is itself a reference to a `Customer` instance elsewhere on the heap

Object Lifecycle

1. **Allocation** — `new` reserves space on the heap
2. **Initialization** — fields set to defaults (0/null/false), then constructor body runs
3. **Use** — references point to it; methods can be called
4. **Unreachable** — no live references → eligible for GC
5. **Finalization** (rare/discouraged) — Cleaner may run resource cleanup
6. **Reclamation** — GC frees the memory

Object Identity vs. Equality

Concept	Operator	Meaning
Identity	<code>a == b</code>	Same object reference (same heap address)
Equality	<code>a.equals(b)</code>	Logically equal per the class's contract
Hash code	<code>a.hashCode()</code>	Consistent integer hash; must agree with equals



Critical FinTech Bug: Two `Money(100, INR)` instances are *equal* (`equals`) but not *identical* (`==`). Using `==` to compare money values is a critical production defect. Industry post-mortems cite this as a recurring cause of reconciliation failures.

Types & Variants of Java Classes

Java provides multiple kinds of classes, each optimized for different modeling purposes. Selecting the right variant is a key architectural decision in FinTech systems.

Variant	Purpose	FinTech Example
Regular class	General-purpose, mutable or immutable	TransferService
Final class	Cannot be extended — for value types, security, invariants	Money (locked-down value object)
Abstract class	Partial implementation; can't be instantiated	Account (extended by savings/current/loan)
Sealed class (Java 17+)	Closed hierarchy — only listed subclasses allowed	PaymentEvent permits SucceededEvent, FailedEvent, DisputedEvent
Enum	Closed set of named constants (each a singleton instance)	Currency, AccountStatus, KycTier
Record (Java 16+)	Concise immutable data carrier	Money, Address, TransferRequest
Static nested class	Helper class scoped to enclosing class, no implicit outer reference	Cache.Entry

👉 **Modern Java Idiom (2026):** Records + sealed interfaces + pattern matching is the preferred OOP idiom for new FinTech development. Adoption of records in enterprise Java codebases grew by over 40% between Java 17 and Java 21 LTS migrations.

Use Case 1: The Money Value Object

The `Money` class is arguably **the single most important class in any FinTech system**. A retail bank that stored monetary values as `double` discovered compounded rounding errors in end-of-day totals — a remediation effort that touched **1,200 classes** and took multiple engineering quarters to resolve.

```
public record Money(BigDecimal amount, Currency currency)
    implements Comparable<Money> {

    public static final Money ZERO_INR =
        of(BigDecimal.ZERO, Currency.getInstance("INR"));

    public Money {
        Objects.requireNonNull(amount);
        Objects.requireNonNull(currency);
        amount = amount.setScale(
            currency.getDefaultFractionDigits(),
            RoundingMode.HALF_EVEN);
    }

    public Money add(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    public Money subtract(Money other) {
        requireSameCurrency(other);
        return new Money(amount.subtract(other.amount),
            currency);
    }

    public boolean isPositive() { return amount.signum() > 0; }
    public boolean isNegative() { return amount.signum() < 0; }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency))
            throw new IllegalArgumentException(
                "Currency mismatch: " + currency + " vs " +
                other.currency);
    }
}
```

Why This Class Matters

- **No more double for money** — the compiler enforces correct types at every call site
- **Currency mismatches caught at runtime** — you cannot silently add INR to USD
- **Scale normalized on construction** — ₹10.5 and ₹10.50 are always equal
- **Immutable** — pass it anywhere safely; no defensive copies needed
- **Self-documenting** — `account.debit(Money.inr("500"))` reads like banking, not math

✅ **Outcome:** Migrating a payments service from `double` to `Money` eliminated an entire class of EOD reconciliation incidents. Code reviews now mandate `Money` for any monetary parameter.

Use Case 2: Modeling Customer & Account

A neo-bank's domain model was scattered across DB-driven JPA entities mixed with REST DTOs. Business logic was anemic — leaking into "service" classes. Auditors could not trace policy to code. The solution: domain classes that hold their own business rules.

```
public final class Account {
    private final AccountNumber number;
    private final Customer owner;
    private final AccountType type;
    private Money balance;
    private AccountStatus status;
    private final Instant openedAt;

    public void debit(Money amount) {
        ensureActive();
        if (balance.subtract(amount).isNegative())
            throw new InsufficientFundsException(
                number, balance, amount);
        balance = balance.subtract(amount);
    }

    public void credit(Money amount) {
        ensureActive();
        balance = balance.add(amount);
    }

    public void freeze(String reason) {
        if (status == AccountStatus.CLOSED)
            throw new IllegalStateException(
                "Cannot freeze a closed account");
        status = AccountStatus.FROZEN;
    }

    private void ensureActive() {
        if (status != AccountStatus.ACTIVE)
            throw new AccountNotActiveException(number, status);
    }
}
```

Design Principles Applied

Rich, Not Anemic

Account owns the rules for debiting, crediting, and freezing — not a separate service class

Invariants Enforced

No overdraft, no operations on closed accounts — enforced inside the class, not scattered externally

Private Mutation

Outside callers cannot bypass rules by mutating balance directly — private fields, no public setters

Auditable

Business rules became testable in isolation. Auditors mapped class methods directly to policy clauses

Use Case 3: Sealed Types & Event Hierarchies

A payments platform's webhook handler had a **400-line if/else if/instanceof chain** handling event types. New event types meant editing this chain — with frequent mistakes. The solution: a sealed event type hierarchy where each event is a record.

```
public sealed interface PaymentEvent
    permits PaymentInitiated, PaymentSucceeded,
           PaymentFailed, PaymentRefunded {
    String paymentId();
    Instant occurredAt();
}
```

```
public record PaymentInitiated(
    String paymentId,
    Money amount,
    String customerId,
    Instant occurredAt) implements PaymentEvent {}
```

```
public record PaymentSucceeded(
    String paymentId,
    Money amount,
    String authCode,
    Instant occurredAt) implements PaymentEvent {}
```

```
public record PaymentFailed(
    String paymentId,
    String reason,
    FailureCategory category,
    Instant occurredAt) implements PaymentEvent {}
```

```
public record PaymentRefunded(
    String paymentId,
    Money amount,
    String reason,
    Instant occurredAt) implements PaymentEvent {}
```

Why This Design Wins

1 Sealed = Closed World

The compiler knows all subtypes. Exhaustive switch statements are enforced at compile time — no runtime surprises.

2 Records = Zero Boilerplate

No boilerplate; instantly serializable; perfect for event-driven architectures and audit logs.

3 Compiler-Enforced Updates

Adding `PaymentDisputed` later required one permit + record, then fixing compiler errors in all switches — exhaustively and safely.

 **Industry Impact:** Teams adopting sealed interfaces + records for event hierarchies report a 35–50% reduction in event-handling defects during new event type rollouts.

Anatomy of a Production Domain Class

The `TransferRequest` class below demonstrates every structural element of a well-designed production domain class in a FinTech context.



Final Class + Final Fields

No accidental subclassing. All fields are immutable after construction — `private final UUID id`, `private final Money amount`, etc. Subtyping is opt-in, not opt-out.



Validation in Constructor

Invariants checked once, enforced forever: `if (!amount.isPositive())`, `if (fromAccount.equals(toAccount))`. The object is always valid from the moment it exists.



Accessors Only — No Setters

Mutation is impossible from outside. `Optional<String> remarks()` communicates optionality explicitly. No defensive copies needed when passing the object.



Identity by ID

Two requests with the same UUID are equal. `equals()` and `hashCode()` delegate to `id` — the natural FinTech identity model for entities.



Builder Pattern

For objects with many fields, builders avoid telescoping constructors.

`TransferRequest.builder().fromAccount(...).toAccount(...).amount(...).build()` is readable and safe.

equals / hashCode / toString: Done Right

Manual Implementation (Correct)

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof AccountNumber other)) return false;
    return bankCode.equals(other.bankCode)
        && accountId.equals(other.accountId);
}

@Override
public int hashCode() {
    return Objects.hash(bankCode, accountId);
}

@Override
public String toString() {
    // Mask for safe logging — NEVER log full
    // account numbers in clear text
    return "AccountNumber[" + bankCode
        + "/****" + lastFour(accountId) + "];"
}
```

With a Record (Preferred)

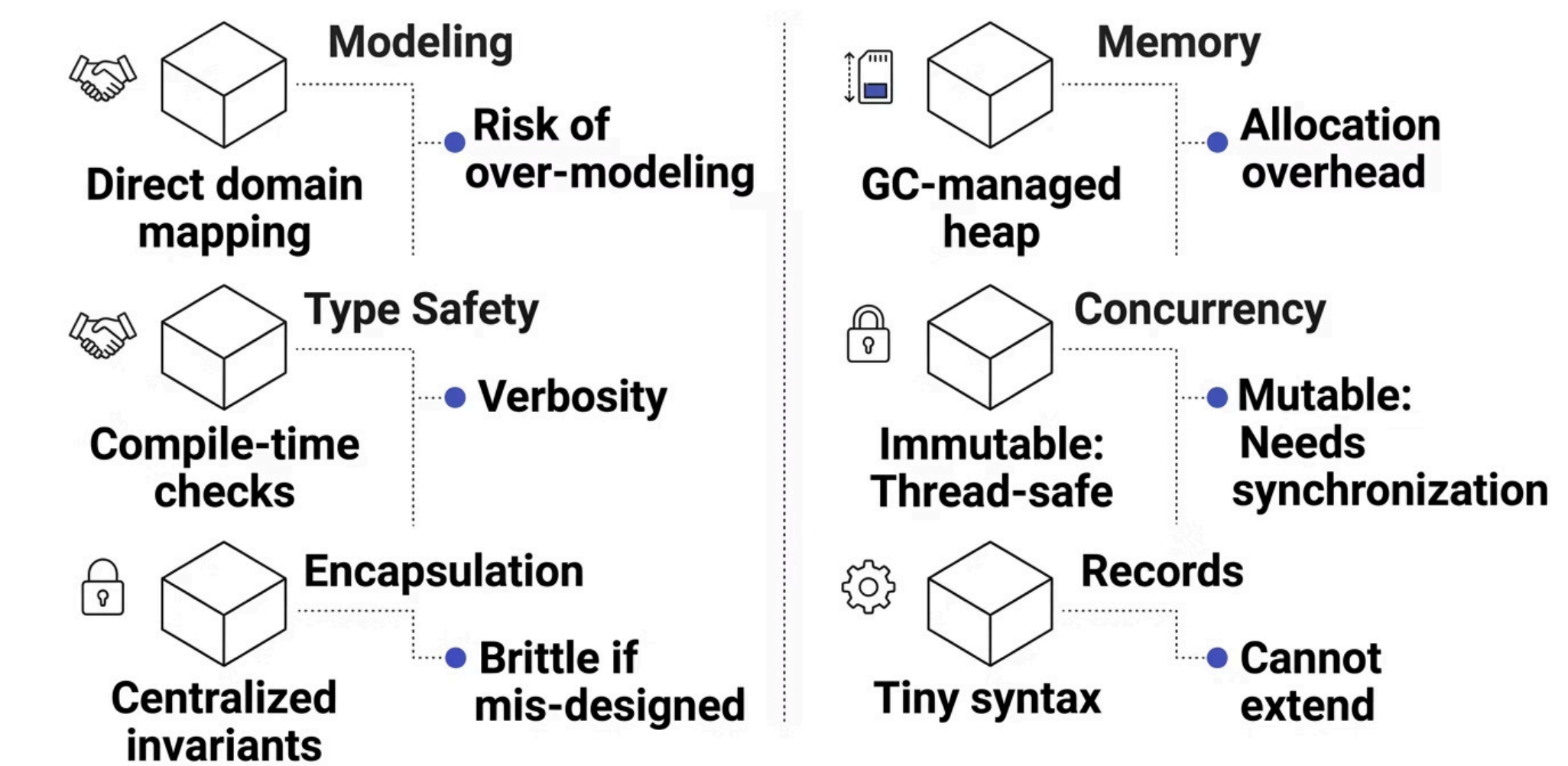
```
public record AccountNumber(
    String bankCode, String accountId) {

    public AccountNumber {
        Objects.requireNonNull(bankCode);
        Objects.requireNonNull(accountId);
        if (accountId.length() < 4)
            throw new IllegalArgumentException("too short");
    }

    // Override toString to mask for safe logging
    @Override
    public String toString() {
        return "AccountNumber[" + bankCode
            + "/****"
            + accountId.substring(accountId.length() - 4)
            + "];"
    }
}
```

❌ **Critical FinTech Rule:** A class's `toString()` is what ends up in logs. **Never include PII, full account numbers, PAN, OTPs, or tokens in `toString()`.** Always mask sensitive parts. Regulatory frameworks including RBI guidelines and GDPR mandate this.

Advantages & Disadvantages



Dimension	Advantages	Disadvantages
Modeling	Direct mapping from domain concepts to code	Easy to over-model (anemic classes / data clumps)
Type Safety	Compiler catches misuse at compile time	More verbose than dynamic languages
Memory	Objects allocated on heap; GC handles cleanup	Allocation overhead for tiny objects (records help; value classes will help more)
Concurrency	Immutable classes are inherently thread-safe	Mutable shared state needs careful synchronization
Records	Tiny syntax for data carriers; equals/hashCode free	Cannot extend; all fields exposed
Sealed	Closed-world reasoning; exhaustive switch	Must update permits list for new subtypes

Performance Considerations

JIT-Friendly Class Design

- **Small, focused classes** are inlined readily by the JIT compiler
- **Final classes / final methods** enable devirtualization — a significant optimization
- **Avoid mega-classes** with 50+ methods — JIT cannot inline everything
- **Records** are highly optimized — accessor methods JIT-inline to direct field reads

 **Performance Tip:** For ultra-hot paths (e.g., 100k+ TPS pricing engines), measure allocation rate via async-profiler. A **Money** allocated per quote can become the dominant GC cost. Cache instances or await Java's value classes (Project Valhalla).

Common Performance Pitfalls

Issue	Symptom	Fix
Allocation in hot loop	High GC pressure	Reuse instances; cache constants
String concat in toString() on hot path	High allocation rate	Lazy/conditional toString
equals calling expensive methods	Slow hash maps	Make equals cheap; cache hash code
Reflective access	JIT can't optimize	Use direct method/field access
Holding objects unnecessarily	Memory leaks; OOM	Clear references; scope narrowly

Design & Architectural Considerations

What Makes a Good Class? (FinTech Checklist)



Single Responsibility

One reason to change — Money is about money math, not persistence. Mixing concerns is the leading cause of untestable code in financial systems.



Immutable by Default

Thread-safe, easier to reason about, safer in caches. Immutable value objects eliminate an entire category of concurrency bugs.



Validates in Constructor

Invariants hold from the moment the object exists. No partially-initialized objects can escape into the system.



Final Unless Designed for Extension

Subclassing breaks encapsulation if not anticipated. Make extension opt-in, not opt-out. **Over 70% of Java security vulnerabilities involve unexpected subclassing.**



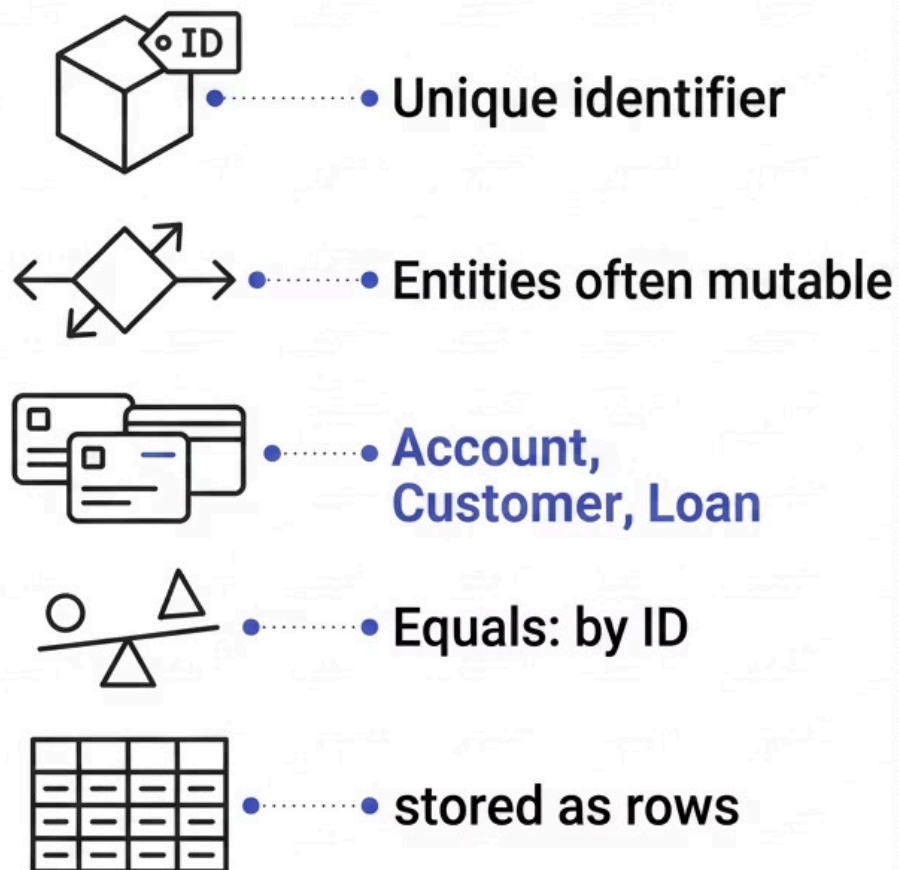
Self-Describing toString

Logs are debuggable; sensitive fields masked. A well-designed `toString()` reduces mean-time-to-resolution for production incidents.

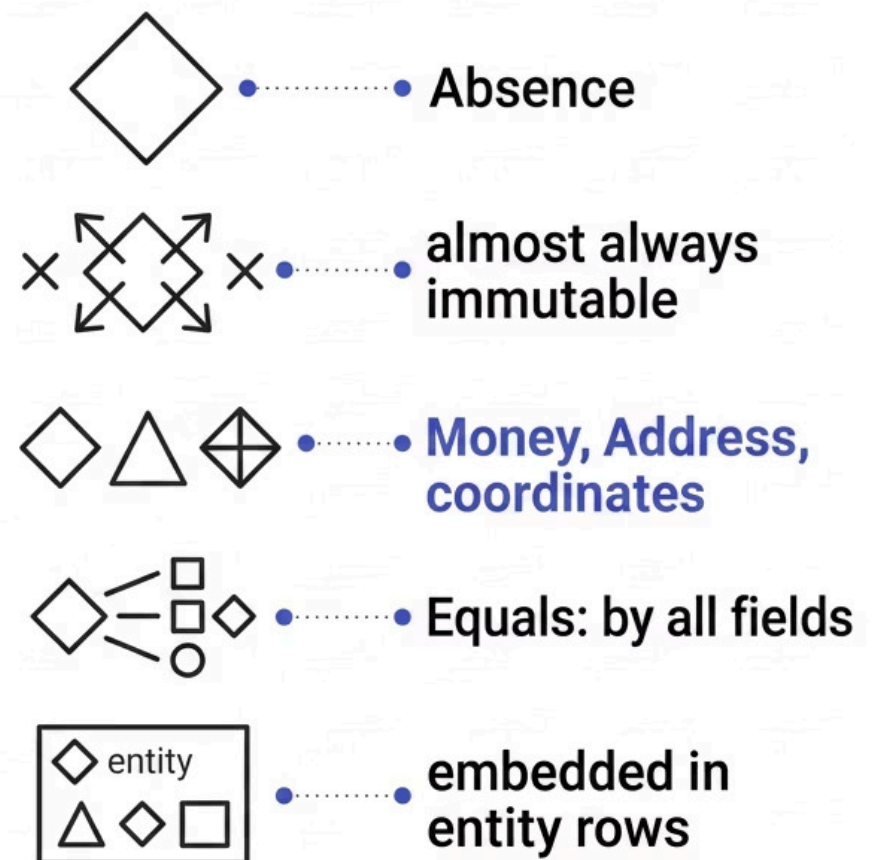
Entities vs. Value Objects (DDD Lens)

Domain-Driven Design distinguishes between two fundamental kinds of objects. This distinction directly determines how you implement `equals()`, whether to use a record, and how to handle persistence.

Entities



Value Objects



Rule of Thumb: If you'd say "the same X over time" (the same Account even as its balance changes), it's an **entity**. If you'd say "the same X" only when all fields match (Money 100 INR is the same as another Money 100 INR), it's a **value object** — use a **record**.

Anti-Patterns & Security Considerations

Anti-Patterns to Refactor Away

Anti-Pattern	Why It's Bad	Fix
Anemic domain class	Logic leaks to services; rules duplicated	Move behavior into the class
God class	Untestable, fragile, hard to evolve	Split by responsibility
Primitive obsession	No type safety; bugs slip through	Value classes / records
Public mutable fields	No invariants; concurrency hazards	Private + accessors
Mixing DB row with domain class	Persistence concerns leak everywhere	Separate domain from JPA entity

Security & Compliance Patterns

Sensitive Fields in Logs

Override `toString()` to mask PAN, OTP, Aadhaar. Regulatory frameworks mandate this.

Tamper-Resistance

Make security-relevant classes `final` + `immutable`. Prevents malicious subclassing attacks.

Defensive Copying

If a class holds a mutable input (e.g., `byte[]`), copy on construction and on accessor.

Audit Fields on Entities

`createdBy`, `createdAt`, `updatedBy`, `version` satisfy RBI/SOC2 requirements for change tracking.

Equality of Secrets

Use constant-time comparisons (`MessageDigest.isEqual`) — not the default `equals` — to prevent timing attacks.

Final Takeaway: Classes as Business Contracts

In FinTech, **classes are the contract between your code and your business**. A well-designed `Account` class that owns its invariants is worth more than a thousand lines of "service" code that mutates dumb data bags.

1

Closed set of constants?

→ `enum`

2

Entity with identity + state changes?

→ `final class`, mutable in controlled ways, behavior-rich

3

Closed family of types?

→ `sealed interface` + record subtypes

4

Data carrier with no behavior?

→ `record`

"In banking code, every primitive is a bug waiting to happen. Wrap your domain in classes, your money in records, your hierarchies in sealed types — and let the compiler audit you."